

Python Operators

```
In [1]: # Arithmetic operators
# +, -, *, /, **, //, %

print(10 + 3) # 13
print(10 - 3) # 7
print(10 * 3) # 30
print(10 / 3) # 3.333....
print(10 ** 3) # pwo = 10*10*10 = 1000
print(10 // 3) # Q => 3
print(10 % 3) # R => 1
```

```
13
7
30
3.3333333333333335
1000
3
1
```

```
In [6]: # Assignment operators
# =, +=, -=, /=, *=, **=, //=, %=
```

```
a = 10
print(a)

a += 5 # a = a + 5 =15
print(a)

a *= 2 # a = a * 2 = 30
print(a)

b = 0
b += 20
print(b)
```

```
10
15
30
20
```

```
In [1]: # Comparison operators (True or False)
# ==, !=, >, <, <=, >=
```

```
print(10 == 10) # True
print(10 != 10) # False
print(10 > 10) # False
print(10 >= 10) # True
print(10 < 10) # False
print(10 <= 10) # True
```

```
True
False
False
True
False
True
```

```
In [ ]: # Logical operators
# and, or, not

print('----AND----')

print(10 == 10 and 6 > 4) # T and T = T
print(10 != 10 and 6 > 4) # F and T = F
print(10 == 10 and 6 < 4) # T and F = F
print(10 != 10 and 6 < 4) # F and F = F

print('----OR----')
print(10 == 10 or 6 > 4) # T or T = T
print(10 != 10 or 6 > 4) # F or T = T
print(10 == 10 or 6 < 4) # T or F = T
print(10 != 10 or 6 < 4) # F or F = F

print('----Not----')

print(not 10==10) # F
print(not 10!=10) # T
```

```
----AND----
True
False
False
False
----OR----
True
True
True
False
----Not----
False
True
```

Python Collection

List

is a collection which is ordered and changeable.

Allows duplicate members. Denoted by **[]**

```
In [ ]: #
mylist = [10,20.55,"Hello",True,10,"Hello",20.55,False,True,10,20.55,True]

print(mylist,type(mylist))
```

```
print(mylist[5])
print(mylist[8])

mylist[8]="Welcome"
print(mylist)
```

```
[10, 20.55, 'Hello', True, 10, 'Hello', 20.55, False, True, 10, 20.55, True, 10, 10] <class 'list'>
Hello
True
[10, 20.55, 'Hello', True, 10, 'Hello', 20.55, False, 'Welcome', 10, 20.55, True, 10, 10]
[10, 20.55, 'Hello', True, 10, 'Hello', 20.55, False, 'Welcome', 10, 20.55, True, 10, 10, 'Chakaravarthi']
```

Tuple

is a collection which is ordered and unchangeable.

Allows duplicate members. Denoted by **()**

```
In [17]: #           0   1   2   3   4   5   6   7   8   9   10  11
mytuple = (10,20.55,"Hello",True,10,"Hello",20.55,False,True,10,20.55,True,10,10)
print(mytuple,type(mytuple))

print(mytuple[4])
print(mytuple[7])

mytuple[7]="Python"
print(mytuple)
```

```
(10, 20.55, 'Hello', True, 10, 'Hello', 20.55, False, True, 10, 20.55, True, 10, 10) <class 'tuple'>
10
False
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[17], line 8
      5 print(mytuple[4])
      6 print(mytuple[7])
----> 8 mytuple[7]="Python"
      9 print(mytuple)

TypeError: 'tuple' object does not support item assignment
```

Set

is a collection which is unordered and unindexed.

No duplicate members. Denoted by **{}**

```
In [1]: myset = {100,"hai",100,"hello","hai",50.55,100,"hello",50.55,True,False}
print(myset,type(myset))
```

```
{False, True, 50.55, 'hai', 100, 'hello'} <class 'set'>
```

Dictionary is a collection that is unordered, changeable (mutable), and stores data in key-value pairs. Keys must be unique (no duplicate keys). It is denoted by curly braces {} using the format **{key: value}**.

```
In [8]: mydict = {
        "Name": "Jeevan",
        "Cours": "Python",
        "Age": 15,
        100: 50.55,
        40.55: True,
    }
    print(mydict, type(mydict))

    print(mydict["Name"])
    mydict["Age"] = 16
    mydict["City"] = "Chennai"
    print(mydict)
```

```
{'Name': 'Jeevan', 'Cours': 'Python', 'Age': 15, 100: 50.55, 40.55: True}
<class 'dict'>
Jeevan
{'Name': 'Jeevan', 'Cours': 'Python', 'Age': 16, 100: 50.55, 40.55: True,
 'City': 'Chennai'}
```

Mutable vs Immutable Objects

Immutable (*int, float, bool, str, tuple*)

- Same data stores in Same location
- Different Data stores in Different location

```
In [12... a = 100
b = 100
c = 101
print(a, type(a), id(a))
print(b, type(b), id(b))
print(c, type(c), id(c))
```

```
100 <class 'int'> 140729209669144
100 <class 'int'> 140729209669144
101 <class 'int'> 140729209669176
```

```
In [13... x = (10, 20, 30, 40, 50)
y = (10, 20, 30, 40, 50)

print(x, type(x), id(x))
print(y, type(y), id(y))
```

```
(10, 20, 30, 40, 50) <class 'tuple'> 1938168950480
(10, 20, 30, 40, 50) <class 'tuple'> 1938168952640
```

Mutable

- Same data stores in Different location
- Different Data stores in Different location

```
In [14... x = [10,20,30,40,50]
y = [10,20,30,40,50]

print(x,type(x),id(x))
print(y,type(y),id(y))

[10, 20, 30, 40, 50] <class 'list'> 1938169258752
[10, 20, 30, 40, 50] <class 'list'> 1938154956224
```

```
In [18... # Identity operators (True or False)
# is, is not

a = 100
b = 100

print(a != b) # "==" => Value => True
print(a is not b) # "Is" => location => True

False
False
```

```
In [ ]: x = (10,20,30,40,50)
y = (10,20,30,40,50)

print(x != y)
print( x is y)
```

```
In [25... # Membership operators (True or False)
# in, not in

print('e' in "Hello world")

x = [10,20.55,"hello world"] # item by item

print("hello world" not in x)

True
False
```

```
In [ ]:
```